

# Informe Final

## 1. Portada

- Nombre del proyecto: Sistema de Gestión de Inventarios para Pequeñas Empresas orientado a Restaurante.
- Integrantes del equipo: (Grupo 3: Los Sin Nombre) Miguel Aliaga, Esteban Guzman, Fabian Paredes.
- Asignatura: Diseño de Software - Profesor: Ernesto Pereiras García.
- Universidad de Talca - Facultad de Ingeniería.

|                                      |          |
|--------------------------------------|----------|
| <b>Informe Final.....</b>            | <b>1</b> |
| <b>1. Portada.....</b>               | <b>1</b> |
| 2. Justificación del Proyecto.....   | 3        |
| 3. Objetivos.....                    | 3        |
| 4. Requerimientos del Sistema.....   | 4        |
| 5. Modelado del Sistema.....         | 5        |
| 6. Patrones de Diseño Aplicados..... | 7        |
| 7. Arquitectura del Sistema.....     | 9        |
| 8. Prototipo Navegable.....          | 9        |
| 9. Modelo Computacional.....         | 10       |
| 10. Conclusiones y Reflexión.....    | 12       |
| 11. Bibliografía y Referencias.....  | 13       |

## 2. Justificación del Proyecto

### - Contexto y Problema

La gestión de un restaurante involucra una alta complejidad operativa, especialmente en el control de su inventario. Frecuentemente, los establecimientos, en particular los de menor tamaño o en vías de crecimiento, manejan su stock de ingredientes, platos y bebidas mediante métodos manuales o sistemas genéricos que no están adaptados a la naturaleza rápida y dinámica de la gastronomía. Esto genera problemas críticos: discrepancias entre el stock físico y el registrado, pérdidas económicas por merma de productos perecibles, quiebres de stock en horarios de alta demanda y dificultades para rastrear la relación con los proveedores.

### - Solución Propuesta y Propósito

Para resolver estas deficiencias, se propone el desarrollo de una aplicación web centralizada para la gestión integral del inventario. El propósito de este sistema es automatizar y asegurar la trazabilidad del ciclo de vida de los productos: Desde su ingreso a través de las compras a proveedores, hasta su salida mediante el registro de ventas. La plataforma integrará un sistema de roles diferenciados (Administrador y Empleado), garantizando que las operaciones críticas —como la modificación de stock o la gestión de proveedores— se realicen bajo los permisos correspondientes.

### - Relevancia Tecnológica y Social

Tecnológicamente, el proyecto permite aplicar arquitecturas robustas (cliente-servidor monolítico) y patrones de diseño (como el uso de Observer para el control de alertas de stock y Proxy para la validación de ventas). Social y comercialmente, la herramienta aporta un valor significativo al rubro gastronómico, ya que empodera a los dueños y administradores con datos precisos en tiempo real, optimizando sus recursos, reduciendo el desperdicio de alimentos y mejorando la eficiencia general del servicio al cliente.

## 3. Objetivos

### Objetivo General:

Desarrollar una aplicación web centralizada bajo una arquitectura cliente-servidor que optimice la gestión de inventario, ventas y proveedores de un restaurante, asegurando el control de stock en tiempo real y la administración segura de la información mediante un sistema de roles y permisos.

### Objetivos Específicos:

- 1.- Diseñar e implementar una base de datos relacional que estructure eficientemente las entidades principales del negocio: Usuarios (roles y permisos), Productos, Proveedores, Ventas y Compras.
- 2.- Desarrollar un módulo de autenticación y autorización que controle el acceso al sistema, diferenciando las vistas y operaciones permitidas para los roles de Administrador y Empleado (Proxy de Venta).
- 3.- Construir un módulo de gestión de inventario que permita registrar, actualizar y eliminar productos y proveedores, manteniendo un historial de transacciones.

- 4.- Implementar un sistema automatizado de control de stock utilizando el patrón de diseño Observer, capaz de descontar productos tras cada venta y generar alertas tempranas cuando los niveles de inventario sean críticos.
- 5.- Diseñar y exponer una API RESTful en el backend que permita una comunicación fluida, escalable y en formato JSON con la interfaz web del cliente.

## 4. Requerimientos del Sistema

- **RF-01 - actualizar**

**Declaración:** El sistema deberá permitir actualizar la cantidad de productos en el stock.

**Criterios:** Acceso  $\leq 2$  segundos ; mensaje “stock actualizado” cuando se registre una venta.

**Verificación:** Prototipo con validación de actualización aceptada.

**Evidencia:** Reporte de actualización de stock y captura de la alerta de bajo stock.

- **RF-02 - consultar**

**Declaración:** El sistema deberá permitir mostrar lista de productos en stock.

**Criterios:** Acceso  $\leq 2$  segundos ; mensaje “lista de productos - día/hora” cuando se ingresa a la interfaz.

**Verificación:** Prototipo con lista de productos en stock por categoría.

**Evidencia:** Captura de interfaz con la lista de productos en stock.

- **RF-03 - agregar**

**Declaración:** El sistema deberá permitir agregar un producto

**Criterios:** Acceso  $\leq 2$  segundos ; mensaje “producto agregado” cuando se agregue un producto.

**Verificación:** Prototipo con formulario a rellenar de un producto.

**Evidencia:** Captura de formulario y mensaje de confirmación.

### NF-UX-01 – (Usabilidad) Flujo de venta rápido

- **Declaración:** El sistema deberá permitir a un empleado registrar una venta completa con mínima carga cognitiva, desde el menú principal hasta la confirmación del pago.
- **Métrica/Umbral:**  $\leq 5$  clics o acciones desde la pantalla de inicio de sesión hasta confirmar Venta.
- **Verificación y Evidencia:** Cognitive walkthrough sobre prototipo (Figma). Mapa de navegación anotado con conteo de acciones o clicks.

### NF-MN-01 – (Mantenibilidad) Documentación interna del código

- **Declaración:** Cada clase y método público del sistema deberá contar con documentación que describa su propósito, parámetros y valor de retorno, para facilitar la comprensión del sistema.
- **Métrica/Umbral:**  $\geq 80\%$  de los métodos públicos documentados.
- **Verificación y Evidencia:** Revisión manual de clases principales en la inspección del código.

### NF-MD-02 – (Modularidad) Extensibilidad del sistema de roles y permisos

- **Declaración:** El sistema de permisos deberá estar diseñado de modo que agregar un nuevo rol o un nuevo permiso requiera cambios de forma rápida y sin afectar otras funciones del sistema.
- **Métrica/Umbral:** Incorporar un nuevo rol o permiso deberá implicar modificaciones en  $\leq 2$  clases del sistema.

- **Verificación y Evidencia:** Se documenta paso a paso la incorporación del rol en el sistema y se listan todos los archivos modificados.

## 5. Modelado del Sistema

### Diagrama de clases UML

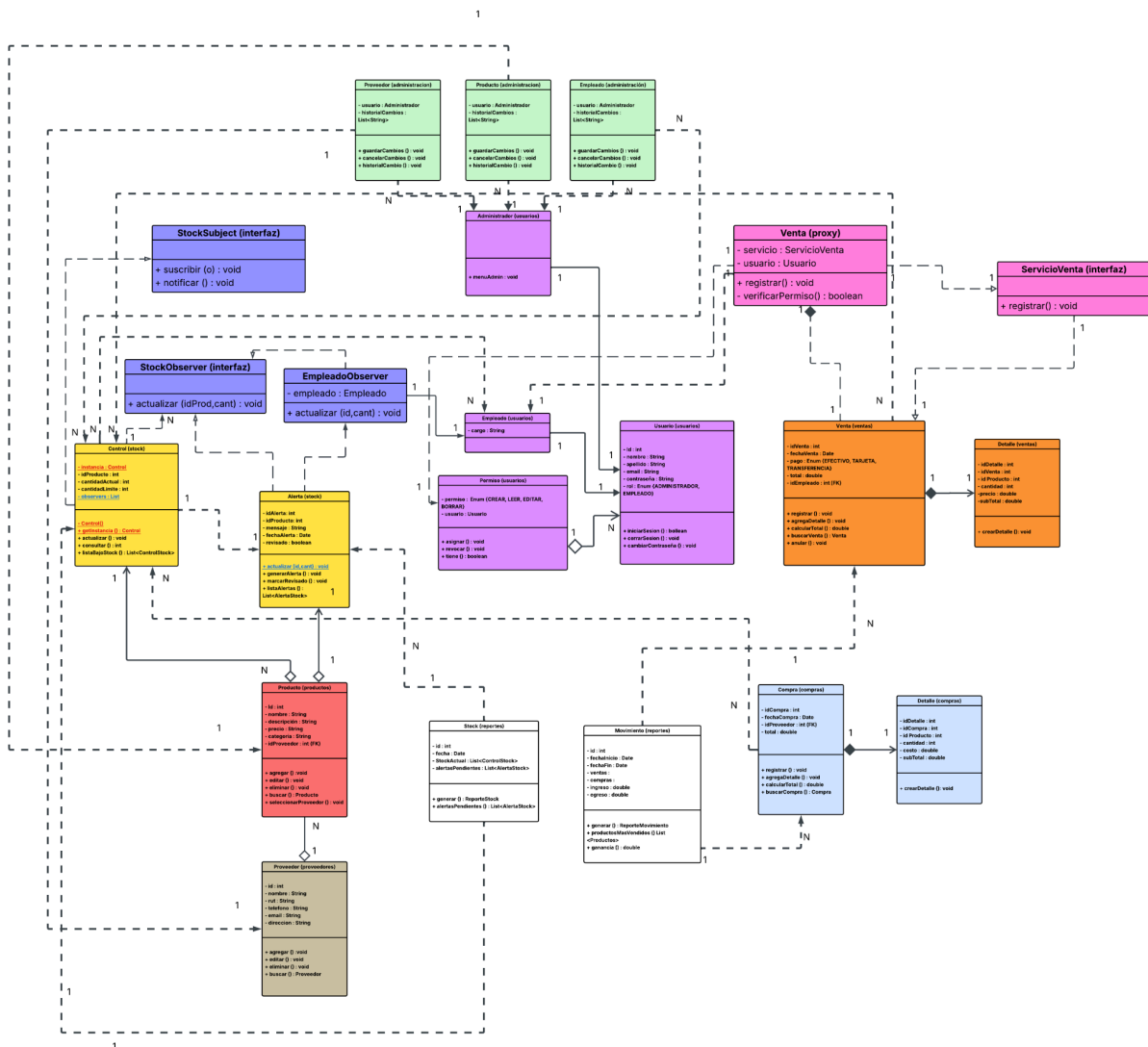


Diagrama de casos de uso: se describe el manejo de productos entre administrador y empleado.

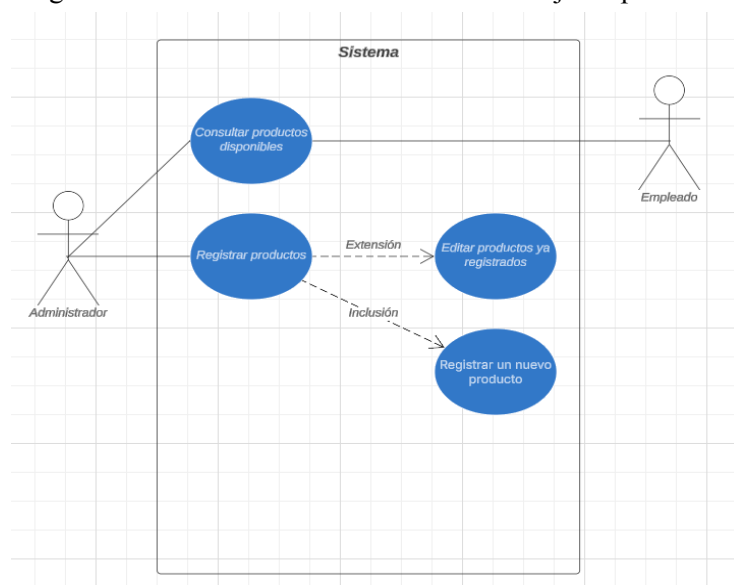
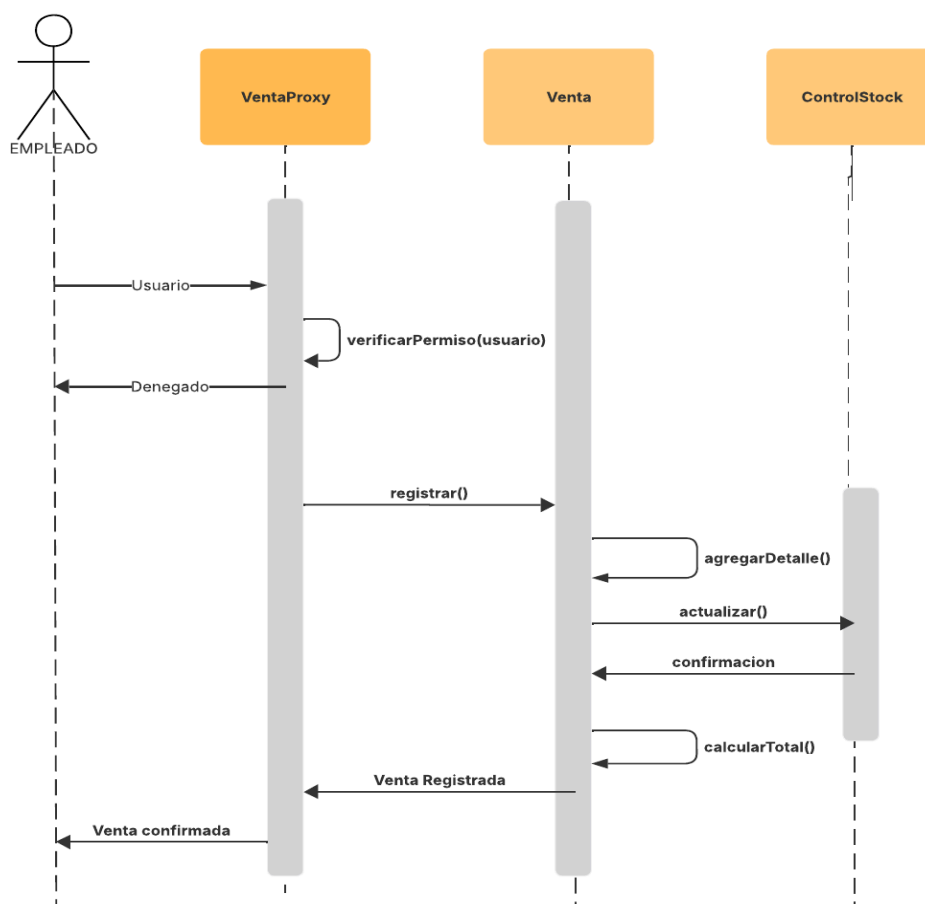
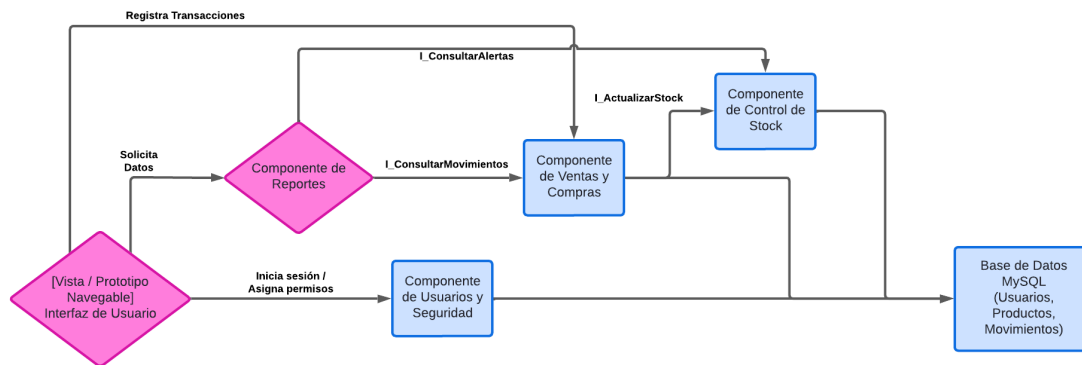


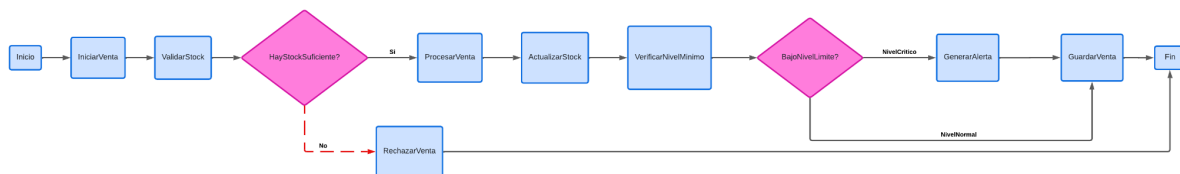
Diagrama de secuencia: Registro de Venta con Proxy



## Diagrama de componentes: Arquitectura del Sistema



## Diagrama de actividades: Flujo de Control de Stock y Alertas

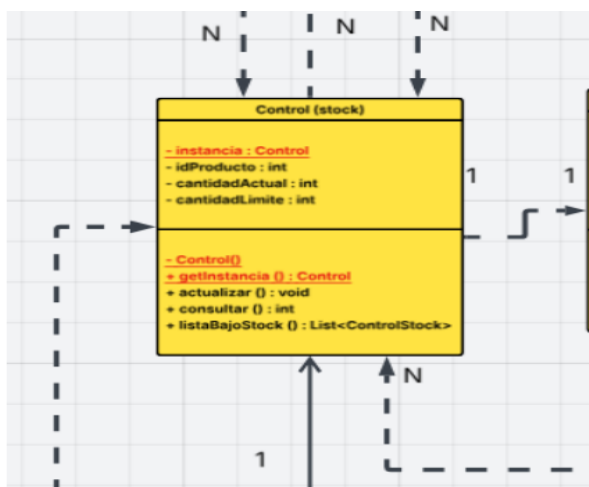


## 6. Patrones de Diseño Aplicados

### Singleton

Se implementó el patrón Singleton en la clase Control (stock) para centralizar el estado del inventario. Esto asegura que cuando Ventas reduce el stock, el módulo de Alerta vea ese cambio inmediatamente, ya que ambos consultaban la misma instancia. Generando una mayor libertad al momento de requerir alguna modificación dentro de esta clase, sin afectar el resto del programa.

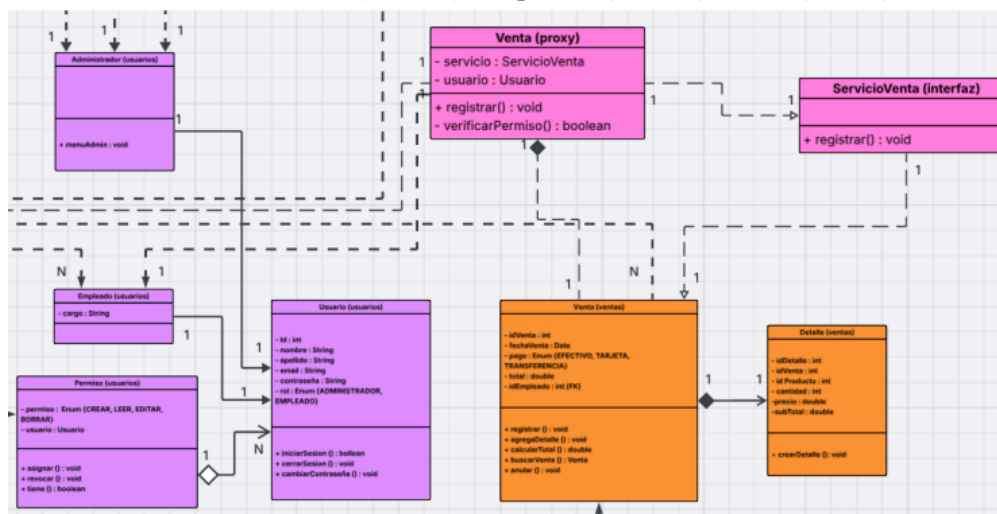
Clases involucradas: Control (stock), Producto (productos), Alerta (stock), Compra (compra), Venta (venta), Stock (reportes), Empleado (usuarios), Empleado (administración)



## Proxy

En el desarrollo del diagrama UML realizado con anterioridad se encontró una modificación factible para la implementación de proxy debido a que este daba pie al desarrollo principal de este patrón, por lo que se decidió realizar la completación de este. El Proxy valida el rol del usuario antes de permitir que la clase real (Venta) ejecute el registro.

Clases involucradas: Permiso (usuario), Empleado (usuario), Venta (ventas).

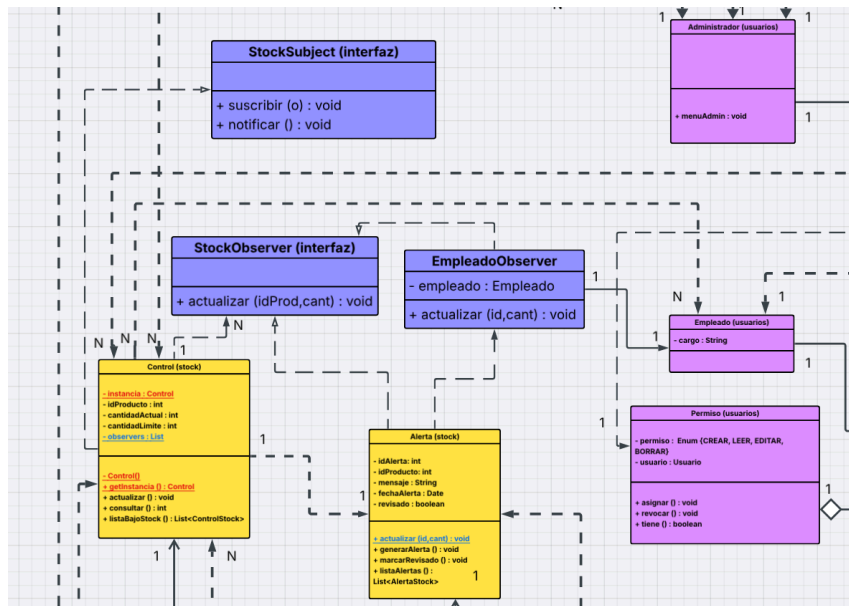


## Observer

Se Implementó el patrón Observer y permite que la clase Control notifique automáticamente a la clase Alerta cuando un producto alcanza su stock mínimo, evitando que el sistema tenga que consultar el inventario constantemente.

Clases involucradas: Control (stock), Alerta (stock), Empleado (usuario).





## 7. Arquitectura del Sistema

Arquitectura Lógica: El sistema se divide en capas claramente definidas para separar responsabilidades. Los principales componentes lógicos son el Frontend (interfaz web), el Servidor (lógica de negocio y API) y la Base de Datos relacional. La comunicación entre el cliente y el servidor se realiza de manera sincrónica mediante peticiones HTTP/REST, utilizando JSON como formato de intercambio de datos.

Arquitectura Física: La infraestructura de despliegue se compone de un servidor centralizado que aloja el backend monolítico y una base de datos relacional (como PostgreSQL o MySQL). Los usuarios (administradores y empleados) acceden al sistema a través de un cliente ligero en navegadores web.

Patrón Arquitectónico: Se seleccionó una arquitectura Cliente-Servidor con un backend Monolítico. Esta decisión se justifica porque el sistema necesita centralizar la información del inventario y las operaciones en tiempo real para evitar inconsistencias transaccionales (por ejemplo, asegurar que una venta descuente el stock inmediatamente). Además, esta arquitectura se alinea con el tamaño del equipo y los tiempos de desarrollo, evitando la sobre ingeniería que supondría arquitecturas más complejas como los microservicios.

## 8. Prototipo Navegable

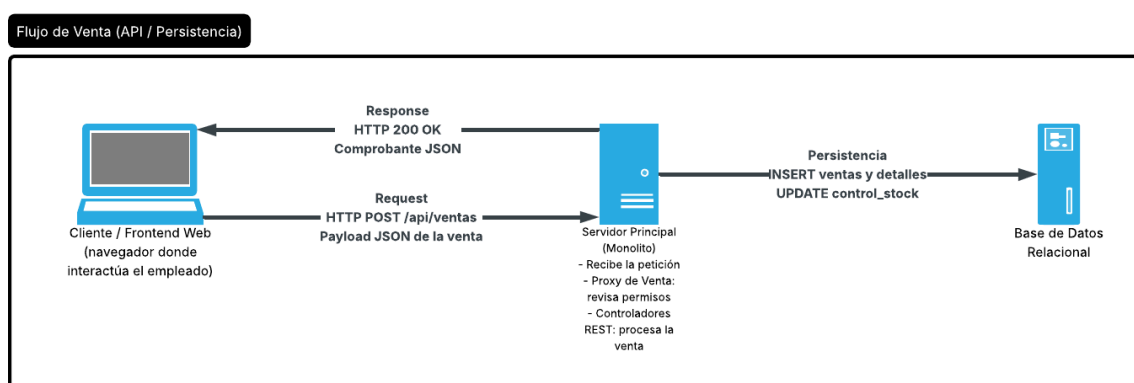
- Herramienta de desarrollo: El diseño de las interfaces se realizó utilizando la herramienta colaborativa **Figma**.
- Descripción del flujo principal: El prototipo simula la experiencia de uso de un **administrador** del restaurante. La pantalla de inicio (**Dashboard**) muestra un resumen del negocio, destacando visualmente las alertas cuando un producto se está quedando sin stock. El flujo

principal de navegación permite al usuario hacer clic en una de estas alertas de "stock crítico", ver los detalles de ese ingrediente o producto, y utilizar un botón para registrar una nueva compra al proveedor, reponiendo así la mercadería.

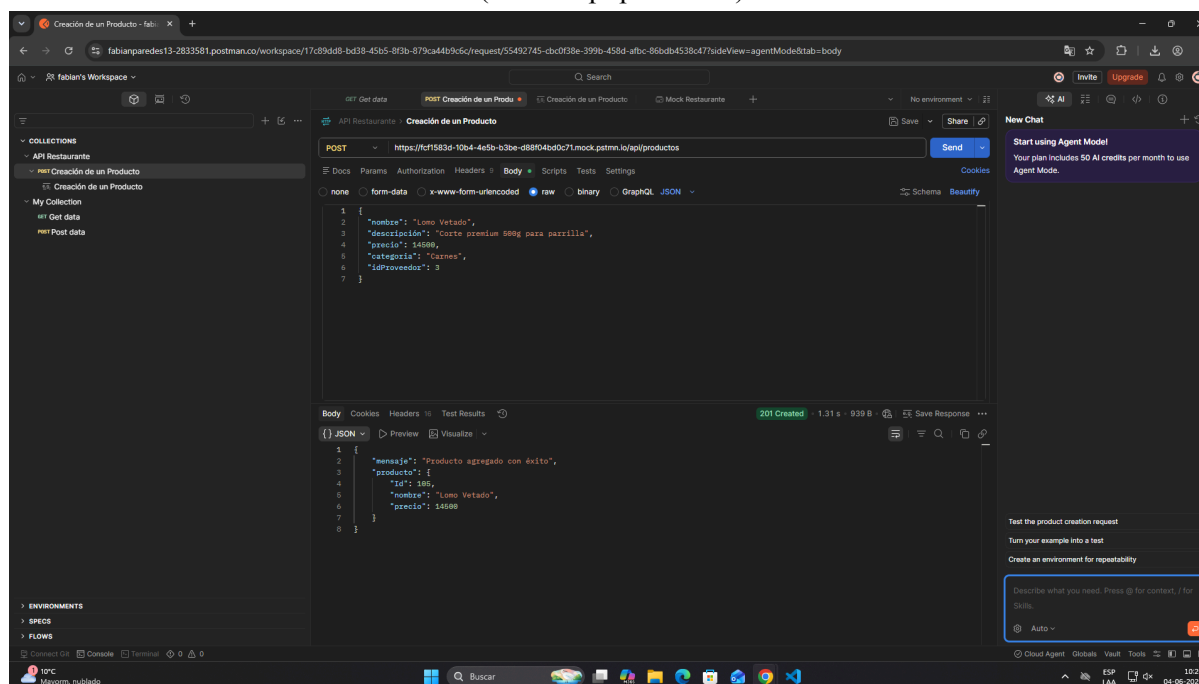
- Enlace público: [\(Figma Grupo 3\)](#).

## 9. Modelo Computacional

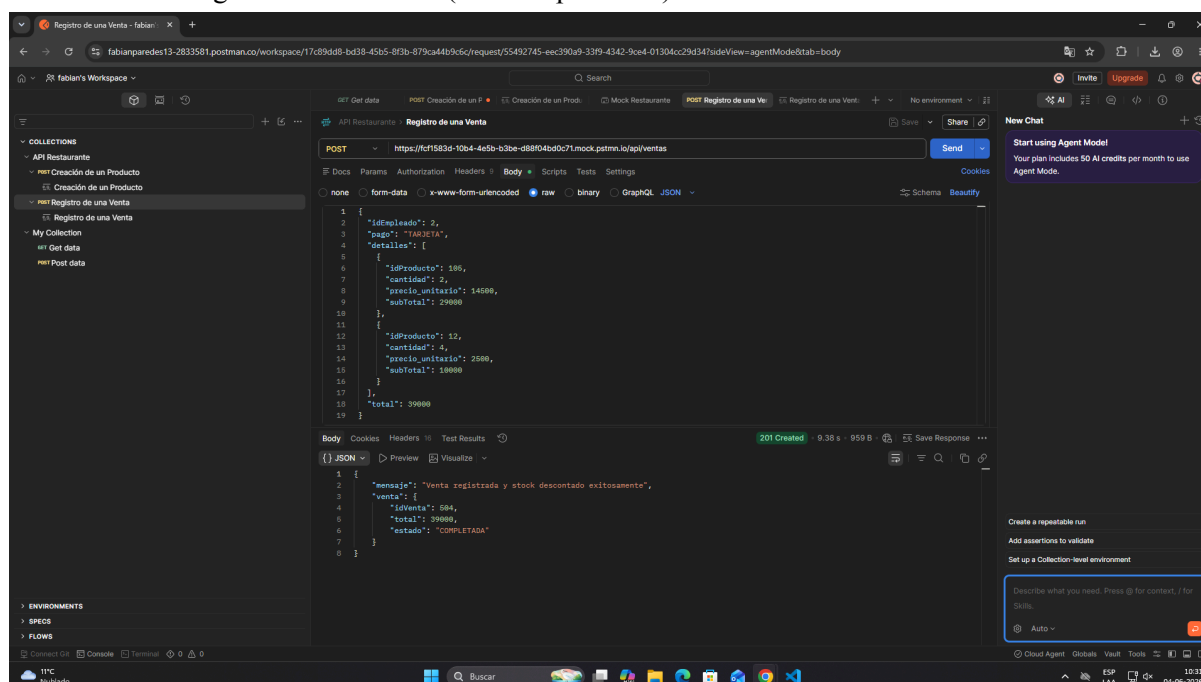
- El cliente (frontend o aplicación móvil). El servidor o backend (API). El flujo de comunicación HTTP (request/response). Los recursos o endpoints principales.



### Simulación de Creación de un Producto (POST /api/productos)



## Simulación de Registro de una Venta (POST /api/ventas)



El sistema procesa la información comunicando la página web del restaurante con el servidor central mediante una API RESTful, intercambiando datos de forma rápida y estructurada utilizando el formato JSON. Las interacciones principales funcionan de la siguiente manera:

- **Gestión de Productos:** Cuando el usuario entra a la sección de inventario, el navegador envía una petición (GET /api/productos) al servidor. El servidor busca la lista en la base de datos y la devuelve a la pantalla. También permite registrar nuevos ingresos o editar precios enviando los datos del formulario (POST o PUT).
- **Registro de Ventas y Seguridad (Proxy):** Al confirmar el pedido de una mesa, se envía la información al servidor (POST /api/ventas). Antes de guardar nada, el sistema pasa por el patrón Proxy, el cual actúa como un guardia de seguridad que revisa si el usuario tiene los permisos correctos (por ejemplo, rol de Empleado o Administrador). Si todo está en orden, la venta se guarda en la base de datos.
- **Actualización y Alertas de Stock (Observer):** Inmediatamente después de guardar la venta, se descuentan los productos vendidos. Gracias al patrón Observer, el sistema vigila si esta resta hace que el producto caiga por debajo de su límite mínimo; de ser así, genera automáticamente una alerta para que el administrador la vea en pantalla (GET /api/stock/alertas).

## 10. Conclusiones y Reflexión

A lo largo del desarrollo de este Sistema de Gestión de Inventarios, el equipo logró organizar y estructurar el código de manera eficiente para resolver los problemas reales de un restaurante.

- Decisiones Clave y Mejoras: Utilizar patrones de diseño ayudó a que el sistema sea mucho más ordenado y seguro. El patrón Singleton fue vital para centralizar el conteo del inventario en un solo lugar, evitando que ocurran errores matemáticos o diferencias de stock. Por otro lado, el patrón Proxy mejoró la seguridad al exigir una validación de permisos antes de permitir que se registre una venta. Además, usar un servidor centralizado (arquitectura monolítica) nos permitió desarrollar de forma más rápida y asegurar que los datos estén siempre actualizados al instante.
- Desafíos Enfrentados: La mayor dificultad técnica fue entender cómo programar el patrón Proxy. Requirió crear nuevas estructuras (como la interfaz ServicioVenta) y separar bien las tareas para que la revisión de permisos no entorpeciera el proceso normal de la venta.
- Riesgos Técnicos y Mejoras Futuras: El principal riesgo de este diseño es que dependemos de un único servidor central; si este se apaga o pierde conexión, el restaurante no podrá registrar ventas ni ver su inventario. En versiones futuras, sería ideal separar la sección que genera los reportes estadísticos, para evitar que esas consultas pesadas pongan lento el sistema de ventas durante los horarios de almuerzo o cena con mucha clientela.

## 11. Bibliografía y Referencias

- Pressman, R. & Maxim, B. Ingeniería del Software: un enfoque práctico.
- Gamma, E., Helm, R., Johnson, R., Vlissides, J. Design Patterns: Elements of Reusable Object-Oriented Software.
- Documentación de herramientas utilizadas (Figma, UML, etc.).
  - UML Grupo 3: [UML 3](#)
  - Figma Grupo 3: [Figma Grupo 3](#)
  - Refactoring Guru: [Patrones de Diseño](#)
  - Docs Postman: [Postman \(API\)](#)

### Informes:

-  Laboratorio 10: Grupo 3 (10)
-  Laboratorio 9 (9)
-  Equipo3: ModeloComputacionalU2 (8)
-  Equipo3: InformeLaboratorioU2 (7)
-  Patrones Creacionales (6)
-  Equipo3\_DS\_InformeU1 (U1)
-  Lab 4 - Equipo 3 (4)

### Diagramas:

1.-

[https://lucid.app/lucidchart/2fa422c3-5479-463c-99c4-35ca9bfbe5eb/edit?viewport\\_loc=-5153%2C-1578%2C11877%2C5993%2C0\\_0&invitationId=inv\\_e8b3ae29-a175-492c-9890-65f284b68183](https://lucid.app/lucidchart/2fa422c3-5479-463c-99c4-35ca9bfbe5eb/edit?viewport_loc=-5153%2C-1578%2C11877%2C5993%2C0_0&invitationId=inv_e8b3ae29-a175-492c-9890-65f284b68183)

2.-

[https://lucid.app/lucidchart/d0c4ed37-e517-498b-9d52-27af2eb50108/edit?viewport\\_loc=-1676%2C-503%2C2175%2C1097%2C0\\_0&invitationId=inv\\_4b9265f8-7bb6-4980-8e8b-022bbd4eb324](https://lucid.app/lucidchart/d0c4ed37-e517-498b-9d52-27af2eb50108/edit?viewport_loc=-1676%2C-503%2C2175%2C1097%2C0_0&invitationId=inv_4b9265f8-7bb6-4980-8e8b-022bbd4eb324)

3.-

[https://lucid.app/lucidchart/4eb3e9c4-64e4-41c2-949c-e3e91201e3b5/edit?viewport\\_loc=-690%2C-182%2C3262%2C1646%2CFDQc8L-t--qg&invitationId=inv\\_a7d18e48-ca6e-47e2-bfe8-db455084eac](https://lucid.app/lucidchart/4eb3e9c4-64e4-41c2-949c-e3e91201e3b5/edit?viewport_loc=-690%2C-182%2C3262%2C1646%2CFDQc8L-t--qg&invitationId=inv_a7d18e48-ca6e-47e2-bfe8-db455084eac)